
EXPERIMENT 5: Unigram Language Model with Laplace Smoothing

AIM:

To build a Unigram Language Model and apply Laplace (Add-1) Smoothing to handle zero-probability words, and compute sentence probability using both unigram and smoothed probabilities.

PROCEDURE:

1. Import numpy and re.
2. Define sample text (two sentences).
3. Define preprocess() to lowercase, remove non-alpha characters, and split.
4. Tokenize text and build vocabulary.
5. Step 4: Compute unigram counts.
6. Step 5: Compute unigram probabilities (count/total).
7. Step 6: Apply Laplace smoothing: $(\text{count} + 1) / (N + V)$.
8. Step 7: Define sentence_probability(sentence, probs): multiply probabilities of tokens; use 1e-6 for OOV.
9. Step 8: Print tokens, vocabulary, unigram probabilities, Laplace probabilities, and sentence probabilities.

CODE:

```
import numpy as np
import re

text = """
Language models learn patterns in text.
Language processing helps computers understand human language.
"""

def preprocess(text):
    text = text.lower()
    text = re.sub(r'^a-z\s', '', text)
    return text.split()

tokens = preprocess(text)
vocab = list(set(tokens))
N = len(tokens)
V = len(vocab)

# Unigram counts
counts = {}
for word in tokens:
    counts[word] = counts.get(word, 0) + 1
```

```
# Unigram probabilities
unigram_probs = {w: counts[w]/N for w in counts}

# Laplace smoothing
def laplace_smoothing(counts, vocab, N, V):
    return {w: (counts.get(w, 0) + 1) / (N + V) for w in vocab}

laplace_probs = laplace_smoothing(counts, vocab, N, V)

# Sentence probability
def sentence_probability(sentence, probs):
    tokens = preprocess(sentence)
    prob = 1
    for word in tokens:
        if word in probs:
            prob *= probs[word]
        else:
            prob *= 1e-6
    return prob

test_sentence = "language models understand language"

print("Tokens:", tokens)
print("\nVocabulary:", vocab)
print("\nUnigram Probabilities:")
for w, p in unigram_probs.items():
    print(w, ":", p)
print("\nLaplace Smoothed Probabilities:")
for w, p in laplace_probs.items():
    print(w, ":", p)
print("\nSentence Probability (Unigram):",
      sentence_probability(test_sentence, unigram_probs))
print("\nSentence Probability (Laplace):",
      sentence_probability(test_sentence, laplace_probs))
```

OUTPUT:

```
Tokens: ['language', 'models', 'learn', 'patterns', 'in', 'text', 'language',
'processing', 'helps', 'computers', 'understand', ...]
Vocabulary: ['patterns', 'learn', 'text', 'in', 'helps', 'models', 'language',
'understand', 'processing', 'human', 'computers']

Unigram Probabilities:
language : 0.23076923076923078
models : 0.07692307692307693
learn : 0.07692307692307693
...

Laplace Smoothed Probabilities:
patterns : 0.08333333333333333
learn : 0.08333333333333333
language : 0.16666666666666666
...

Sentence Probability (Unigram): 0.00031511150169811817
Sentence Probability (Laplace): 0.0001929290123456790122
```

RESULT:

A Unigram Language Model with Laplace Smoothing was successfully implemented. Laplace smoothing redistributes probability mass from frequent words to unseen words, preventing zero probabilities. The sentence probability with Laplace smoothing is slightly lower than unigram but avoids zero-probability issues for out-of-vocabulary words.